Dominio 5 spiegato da zero: gestione del contesto e affidabilità (15%)

Questo dominio è il più "umano": riguarda come un sistema con l'intelligenza artificiale può sbagliare per colpa dei dati che riceve, e come evitarlo. Il filo conduttore è semplice: **Claude decide in base a ciò che vede**. Se l'informazione è stata persa, deformata o nascosta, la decisione sarà sbagliata, anche se il ragionamento è corretto.

> **Prima di iniziare.** Quando incontri → **Pre-corso** trovi il link alla spiegazione completa. Per questo dominio conviene aver letto almeno:
>
> - Token e finestra di contesto (la base di tutta la prima parte);
> - Modello linguistico (LLM), in particolare perché la Claude API "non ricorda" nulla tra una chiamata e l'altra;
> - Richiesta HTTP (codici `200` e `503`) e Rate limit, timeout, retry e backoff;
> - Human-in-the-loop ed escalation e SLA;
> - Allucinazioni.
>
> Per i termini specifici di Claude tieni aperto il Vocabolario.

---

1. Il contesto: la scrivania di Claude

Il **contesto** è tutto ciò che Claude ha davanti in un dato momento: le tue richieste, le risposte precedenti, i risultati degli strumenti, le istruzioni generali. È la sua scrivania.

→ Pre-corso: Token e finestra di contesto

Un dettaglio che chi non programma spesso non sa: Claude **non ha memoria** tra una richiesta e l'altra. Ogni volta il programma gli rimanda l'intera conversazione, dall'inizio. La "scrivania" viene quindi riapparecchiata a ogni turno, e si riempie sempre di più.

→ Pre-corso: Modello linguistico (LLM) e API e REST (cosa significa "stateless", senza stato).

Una scrivania ha una dimensione. Oggi i modelli hanno scrivanie molto grandi (fino a 1 milione di **token**, dove un token è un pezzo di parola: circa 555.000 parole). Ma questo non risolve il problema, per tre ragioni:

1. **Più grande non è gratuito.** Ogni pezzo di scrivania si paga a ogni turno, e la qualità cala quando è troppo piena.
2. **Il problema "perso nel mezzo".** I modelli leggono bene l'inizio e la fine di un testo lungo, e trascurano il centro. Come quando in un libro di 400 pagine ricordi bene il primo e l'ultimo capitolo, meno quelli in mezzo.
3. **Il conteggio è cambiato.** Dai modelli 4.7 in poi, lo stesso testo produce circa il 30% di token in più. Se stimi i costi con vecchi strumenti come `tiktoken`, sbagli. Si usa `messages.count_tokens`, che è il conteggio ufficiale.

Conseguenza pratica del punto 2: le informazioni più importanti (istruzioni chiave, fatti del caso) conviene metterle all'**inizio** o alla **fine**, non annegate a metà di un documento lungo.

Cosa va perso quando il contesto viene riassunto

Quando la conversazione diventa lunga, il sistema la riassume (con il comando `/compact` o in automatico). Un riassunto è una parafrasi, e le parafrasi perdono proprio ciò che conta di più:

- `$149.99` diventa "circa $150";
- `2024-01-15` diventa "metà gennaio";
- `23.7%` diventa "circa il 24%";
- numeri d'ordine, ID cliente, codici di riferimento, messaggi di errore esatti.

Pensa al gioco del telefono senza fili: dopo qualche passaggio, il senso generale resta, ma i dettagli precisi cambiano. E nei sistemi aziendali sono proprio i dettagli (l'importo, il numero di pratica) a contare.

La soluzione: il blocco dei fatti. Estrai i dati importanti in un blocco separato, scritto alla lettera, che il riassunto non tocca. Per esempio:

```
FATTI DEL CASO (non riassumere):
- ID cliente: n. 12345
- Ordine: n. 67890
- Totale: $149.99
- Rimborso richiesto: rimborso completo ($149.99)
```

Regola: un numero esatto non sopravvive a una parafrasi. Va ripetuto identico.

Cosa resta dopo `/compact`

Sopravvive: `CLAUDE.md` (che viene riletto dal disco), il system prompt, i messaggi recenti, la memoria automatica (`MEMORY.md`).

Si perde: i dettagli dei turni precedenti, i risultati specifici degli strumenti, i numeri esatti dell'inizio della conversazione.

Il criterio per ricordarlo: sopravvive ciò che sta **in un file** o **nelle istruzioni fisse**; si perde ciò che esisteva **solo nella conversazione**. Per questo le regole importanti vanno in `CLAUDE.md` (dominio 3) e i fatti importanti in un blocco dei fatti o in un file di appunti.

Ridurre il rumore degli strumenti

Uno strumento che cerca un ordine può restituire 40 campi, quando ne servono 5. Ogni campo in più occupa scrivania. Il filtro va fatto **prima** di restituire il risultato a Claude.

→ Pre-corso: Database e agenti: cosa conta per l'esame

Esempio: per rispondere "dov'è il mio ordine?" servono stato, data di spedizione, corriere e numero di tracciamento. Non servono il codice del magazzino, il peso del pacco, la cronologia dei prezzi e altri 30 campi. Restituirli tutti a ogni chiamata, per decine di chiamate, riempie la scrivania di carta inutile.

---

2. Escalation: quando passare la pratica a una persona

Un'**escalation** è il passaggio di un caso a un operatore umano. Un agente deve sapere quando farlo, e quando invece no.

→ Pre-corso: Human-in-the-loop ed escalation

Sbagliare in entrambe le direzioni ha un costo: troppe escalation intasano gli operatori e rendono inutile l'automazione; troppo poche lasciano i clienti con problemi che l'agente non sa risolvere.

Quando sì

- Il cliente chiede esplicitamente di parlare con una persona. Va rispettato subito, senza ulteriori domande.
- C'è una lacuna nelle regole: una situazione che la policy non copre. L'agente non può decidere da solo.
- Non ci sono progressi dopo tentativi ragionevoli.

Quando no

- **Tono acceso o maiuscole.** Un cliente che scrive "SONO FURIOSISSIMO!!!" per un addebito di $5 può avere un problema semplicissimo. **Emozione non equivale a complessità.**
- **Bassa affidabilità dichiarata dal modello.** Quando Claude dice "non sono sicuro", il numero non è calibrato bene. Non usarlo come motivo per passare il caso.
- **Conversazione lunga.** La durata non dice nulla sulla difficoltà.

Il caso che tutti sbagliano: un cliente **calmo** che chiede come lo SLA (l'accordo sul livello di servizio) interagisce con le regole europee sulla residenza dei dati. Il tono è tranquillo, ma il problema è complesso: qui l'escalation è giusta.

→ Pre-corso: SLA e Dati personali e sensibili (il GDPR). Non serve conoscere queste norme: serve capire che una domanda che incrocia contratto e normativa è fuori dalla portata della policy standard.

Una tabella per ripassare:

Situazione	Escalation?	Perché
"Voglio parlare con una persona"	Sì, subito	Richiesta esplicita
Cliente furioso per un addebito doppio di 5 €	No	Problema semplice, risolvibile con un rimborso
Cliente calmo con una domanda su contratto e normativa europea	Sì	Complessità fuori dalla policy
Conversazione lunga ma che sta progredendo	No	La durata non è un criterio
Tre tentativi falliti, nessun progresso	Sì	Mancanza di progressi

Più clienti con lo stesso nome

Se la ricerca trova più account, **non scegliere quello più probabile**. Chiedi un dato in più per distinguerli: un'email, le ultime quattro cifre del telefono.

Scegliere "il più probabile" può voler dire mostrare i dati di un cliente a un'altra persona, o rimborsare l'ordine sbagliato. Una domanda in più costa pochi secondi.

Il passaggio di consegne

Quando passi un caso a una persona, scrivi un **riepilogo strutturato**: nome, contatto, tipo di problema, ordine, azioni già fatte, azione consigliata, urgenza. La persona non vede la conversazione, quindi il riepilogo è tutto ciò che sa.

```
PASSAGGIO A OPERATORE
Cliente: Mario Rossi (ID 12345) · mario@esempio.it
Problema: rimborso di 750 € per ordine 67890, prodotto difettoso
Già fatto: identità verificata; difetto confermato dalle foto
Bloccato perché: importo sopra il limite automatico di 500 €
Azione consigliata: approvare il rimborso completo
Urgenza: media (cliente in attesa da 2 giorni)
```

È lo stesso principio della delega ai subagenti del dominio 1: chi riceve il caso parte da zero.

---

3. Gli errori: dire la verità su cosa è successo

Quando un sistema fallisce, deve raccontarlo in modo che Claude possa rimediare. Un errore utile contiene:

- il **tipo** di errore;
- il **servizio** coinvolto;
- la **query** tentata;
- i **risultati parziali** eventualmente ottenuti;
- le **alternative** possibili;
- se è **riprovabile**, e dopo quanto tempo.

→ Pre-corso: Richiesta HTTP e Log e audit trail. Il dominio 2 mostra un esempio completo di errore ben scritto.

Tre antipattern da riconoscere

1. **Errore generico**: `{"error": "Operation failed"}`. Non dice niente, Claude non sa cosa fare.
2. **Soppressione silenziosa**: `{"results": []}` quando il motore di ricerca era in realtà spento. Claude risponderà con sicurezza "non ho trovato nulla", e il cliente crederà che l'informazione non esista.
3. **Interruzione del workflow**: se un passaggio va in timeout, si ferma tutto. Meglio continuare con i risultati parziali, segnalando la lacuna.

→ Pre-corso: Fallimento parziale e degradazione controllata

La distinzione chiave

Situazione	Stato	Significato	Cosa dice l'agente
Ricerca riuscita, nessuna corrispondenza	`200` con lista vuota	Non esistono contenuti pertinenti	"Non ho trovato articoli"

Situazione	Stato	Significato	Cosa dice l'agente
Servizio spento	`503` o timeout	La ricerca non è stata eseguita	"Al momento non riesco a cercare, provo un altro approccio"

Un'analogia: chiedi in biblioteca se c'è un libro. La bibliotecaria controlla il catalogo e dice "non c'è": è un fatto. Se il catalogo era spento, la risposta giusta è "non posso controllare adesso": non è la stessa cosa.

Nei sistemi con più agenti

Un subagente che incontra un errore temporaneo lo gestisce da solo: riprova con attese crescenti (per esempio 1 secondo, poi 2, poi 4). Solo se non ci riesce, segnala al coordinatore, dicendo cosa ha tentato, cosa ha ottenuto, che errore è successo e se conviene riprovare.

→ Pre-corso: Rate limit, timeout, retry e backoff (il backoff esponenziale).

Perché non segnalare subito al coordinatore? Perché un piccolo intoppo temporaneo si risolve quasi sempre da solo in pochi secondi; disturbare il coordinatore per ogni singhiozzo riempirebbe il suo contesto di rumore.

---

4. Codebase grandi e sessioni lunghe

Una **codebase** è l'insieme dei file di un progetto software. Lavorare su una codebase grande in una sola sessione lunga fa degradare Claude: comincia a fare riferimento a "schemi tipici" invece che al codice reale, dà risposte incoerenti, dimentica le scoperte precedenti e, peggio, comincia a supporre invece di verificare.

È come un consulente che lavora da dieci ore di fila sullo stesso dossier: a un certo punto inizia a rispondere "di solito si fa così" invece di ricontrollare le carte.

Le contromisure:

- **File di appunti** (scratchpad): un file, per esempio `scratchpad.md`, dove salvi i risultati importanti: "il punto d'ingresso è in `src/auth/middleware.ts` riga 23", "il token scade dopo un'ora", "manca un modo per revocare i token". Il file sopravvive alla compressione: quando serve, Claude lo rilegge.
- **Delegare ai subagenti** le esplorazioni lunghe. Un subagente di tipo `Explore` legge 50 file e restituisce un riassunto di poche righe. La conversazione principale resta pulita.
- `/compact` **proattivo**: non aspettare che il sistema riassuma in modo casuale. Riassumi tu, con un riepilogo strutturato, prima che la qualità cali.
- **Manifesto di recupero**: per workflow lunghi con più agenti, ogni agente salva il proprio stato in un file noto, e il coordinatore tiene un elenco (il manifesto) di chi ha finito, chi è a metà, chi deve ancora partire. Dopo un arresto, il coordinatore riparte da lì senza rifare il lavoro già fatto.

Il principio comune a tutte e quattro: **ciò che conta va scritto fuori dalla conversazione** (in un file, in un riassunto strutturato, nel contesto separato di un subagente), perché la conversazione prima o poi verrà compressa.

---

5. Revisione umana: dove guardare, quanto controllare

Nessun sistema è perfetto, e per le decisioni importanti serve una persona che controlli. Ma non si può controllare tutto. Ecco come farlo bene.

→ Pre-corso: Human-in-the-loop ed escalation

Campionamento stratificato

Non prendere campioni a caso dall'intero insieme. Dividi per rischio:

- rischio alto: controlla il 20%;
- rischio medio: il 10%;
- rischio basso: il 5%.

Così la fatica della revisione va dove serve.

Perché non a caso? Se il 90% dei documenti è a basso rischio, un campione casuale sarà quasi tutto composto da documenti facili, e i pochi casi pericolosi rischiano di non essere mai controllati.

Confidenza per campo, non per tutto

Invece di un unico punteggio per l'intera estrazione, dai un punteggio a ogni campo: il nome del fornitore è sicuro al 99%, il numero di partita IVA al 45%. Così sai quale dato guardare.

Ma attenzione: il punteggio ha senso solo se è **calibrato**. Va verificato su un insieme di dati già corretti. Se il sistema dice 0.9, quel campo deve essere giusto circa nove volte su dieci. Senza questa verifica, il numero è decorativo.

Un'analogia: un meteorologo che dice "70% di probabilità di pioggia" è affidabile se, nei giorni in cui l'ha detto, ha piovuto davvero circa 7 volte su 10. Se ha piovuto 2 volte su 10, il suo 70% non significa nulla, per quanto suoni preciso.

Accuratezza per tipo di documento

Un dato aggregato può ingannare. Un sistema con il 97% di accuratezza complessiva sembra ottimo. Ma se si scompone:

- fatture standard: 99.5%;
- note manoscritte: 45%;
- documenti in lingua straniera: 62%.

Il 97% nasconde due categorie in cui il sistema non funziona. Riporta sempre l'accuratezza **per tipo di documento e per campo**.

→ Pre-corso: OCR. Le note manoscritte vanno male spesso proprio perché il testo di partenza è letto male.

Come può un 97% nascondere un 45%? Perché le fatture standard sono la stragrande maggioranza dei documenti: il loro 99,5% "trascina" la media e copre le categorie più piccole.

---

6. Provenienza: da dove viene ogni informazione

Quando un agente raccoglie informazioni da più fonti, ogni affermazione deve portare con sé la sua origine:

- l'affermazione stessa;
- la prova (in quale pagina, in quale paragrafo);

- l'URL o il documento;
- la data di pubblicazione;
- il tipo di fonte (primaria, cioè il documento originale, o secondaria, cioè un commento).

È come la bibliografia di una tesi: ogni affermazione importante ha la sua nota a piè di pagina, così chi legge può verificarla.

→ Pre-corso: Allucinazioni (le citazioni permettono di verificare) e Embedding, ricerca semantica e RAG.

Nei sistemi multi-agente (dominio 1) la provenienza si perde facilmente: il subagente di ricerca trova la fonte, quello di sintesi la riassume, quello dei report scrive il testo finale. Se a ogni passaggio la fonte non viaggia insieme all'affermazione, alla fine resta un'affermazione senza origine.

I dati in conflitto

Due fonti dicono cose diverse? Non scegliere in silenzio. **Annota il conflitto.** Per esempio: il comunicato stampa dice $2.3B (dati preliminari), il deposito presso la SEC dice $2.28B (dati sottoposti ad audit). Il dato più affidabile è il secondo, ma il primo va comunque segnalato. Chi legge deve poter vedere la discrepanza.

La SEC è l'autorità di vigilanza della borsa americana: i documenti depositati lì sono ufficiali e verificati da revisori, mentre un comunicato stampa è più rapido ma meno controllato.

Il grande errore è scegliere un valore senza dire che c'era un disaccordo. Questo nasconde problemi di qualità dei dati.

Le date

Ogni dato va accompagnato dalle sue date:

- `data_as_of`: a quando si riferisce il dato;
- `source_published`: quando la fonte è stata pubblicata;
- `retrieved_on`: quando l'hai recuperato.

Due tassi di disoccupazione diversi (uno di novembre, uno di dicembre) non sono una contraddizione: sono due fotografie di mesi diversi. Senza date, sembrano contraddirsi.

Nella sintesi

Quando scrivi una sintesi di ricerca:

1. **Distingui i livelli di certezza**: "ben documentato" (confermato da più fonti), "controverso" (le fonti divergono), "fonte singola" (una sola fonte lo dice).
2. **Non reinterpretare**: riporta fedelmente ciò che dice la fonte, comprese le sue riserve.
3. **Scegli il formato in base al contenuto**: tabelle per i dati finanziari, ordine cronologico per gli eventi, elenchi per le specifiche tecniche, confronti affiancati per le affermazioni contestate.

---

7. La gestione del contesto fatta dal server

Fin qui il contesto lo gestisci tu, con blocchi di fatti, file di appunti e subagenti. Ma l'API offre anche tre meccanismi che lavorano **sul server**, cioè dal lato di Anthropic. Sono complementari, non alternativi: ciascuno

risolve un problema diverso.

→ Pre-corso: Client e server. "Lato server" significa che il lavoro lo fa Anthropic quando riceve la richiesta, non il programma dello sviluppatore.

Meccanismo	Cosa fa	Metafora
Modifica del contesto	Elimina vecchi risultati di strumenti o blocchi di ragionamento	Buttare via le ricevute vecchie dalla scrivania: sono sparite per sempre
Compattazione	Riassume la conversazione precedente in un blocco firmato	Fare un verbale delle riunioni passate e archiviare gli appunti
Strumento di memoria	Claude legge e scrive file persistenti, fuori dalla conversazione	Un archivio di faldoni che resta anche quando cambi ufficio

La modifica del contesto

Elimina contenuti senza riassumerli. Si configura per esempio così: quando i token di input superano una soglia, si eliminano i vecchi risultati degli strumenti, ma se ne tengono alcuni recenti. Vanno configurati con cura:

- `clear_thinking` deve comparire **per primo**, se usato insieme a `clear_tool_uses`;
- una soglia `clear_at_least` evita di eliminare quantità trascurabili, che invaliderebbero la cache per nulla;
- la risposta dell'API indica che cosa è stato eliminato, in `context_management.applied_edits`.

→ Pre-corso: Cache. Eliminare qualcosa dalla conversazione cambia il testo inviato, e quindi la parte che veniva riusata dalla cache va rielaborata. Ha senso farlo solo quando si libera molto spazio.

La compattazione

Riassume invece di eliminare. Ci sono due modalità:

- **Su richiesta**: sei tu a decidere quando, mandando una richiesta apposta. Puoi anche tenere alla lettera gli ultimi turni e continuare a lavorare mentre il riepilogo viene scritto.
- **A soglia**: l'API la attiva da sola quando la conversazione supera un certo numero di token (circa 150.000).

La documentazione consiglia la modalità su richiesta, quando è disponibile.

Due precisazioni importanti:

- Le istruzioni del riepilogo (`instructions`) **sostituiscono** quelle predefinite: devi scrivere tu cosa conservare.
- Se il riepilogo non viene prodotto perché la risposta si è fermata (`stop_reason` diverso da `compaction`), la conversazione va avanti e si riprova più tardi. Non si usa un blocco che non esiste.

Le trappole della compattazione

Questo è il punto più insidioso del dominio, perché gli errori **non fanno rumore**:

1. **Passa l'intero `response.content`, non solo il testo.** Se estrai solo il testo e lo rimandi, lo stato della compattazione si perde senza alcun errore. Il problema emerge dopo novanta turni, quando è difficile capire perché.
2. **Il blocco di compattazione va messo per primo** nella richiesta successiva. I messaggi che riassume non vanno lasciati dopo il blocco, altrimenti vengono mandati due volte.
3. **Non mettere il blocco senza i messaggi che segue**: in quel caso Claude non riceve affatto il riepilogo.

Per chi non programma: una risposta di Claude non è solo "testo". È un insieme di **blocchi** (testo, richieste di strumenti, ragionamento, riepilogo di compattazione). `response.content` è l'elenco completo dei blocchi; `response.content[0].text` è solo il testo del primo. Salvare solo il testo è come archiviare una lettera buttando via gli allegati: sembra tutto a posto finché non ti serve un allegato.

→ Pre-corso: JSON (un array è un elenco di elementi: `[0]` indica il primo).

Il thinking e le modifiche ai turni precedenti

Con alcuni modelli recenti (Fable 5.1 e Opus 5.5), i blocchi di ragionamento interno sono validi solo rispetto alla cronologia esatta. Se modifichi i turni precedenti (cancelli un messaggio, ne cambi uno), i blocchi successivi non sono più validi.

La modifica del contesto e la compattazione lato server **non contano come modifiche**: il sistema verifica la conversazione così come l'hai inviata. Per questo è più sicuro spostare la riduzione del contesto sul server.

Per cambiare le istruzioni generali (system) o gli strumenti a metà conversazione, senza rompere il thinking, si aggiunge un messaggio con `role: "system"` dopo l'ultimo turno dell'utente, senza toccare `system` e `tools`.

Lo strumento di memoria

È uno strumento che **Claude** usa per leggere e scrivere file persistenti. Anthropic definisce l'interfaccia; tu implementi le operazioni sui file. Se lo combini con la modifica del contesto, Claude riceve l'avviso di salvare le informazioni importanti **prima** che i risultati vengano eliminati. È la versione ufficiale dei file di appunti.

"Anthropic definisce l'interfaccia, tu implementi le operazioni" significa: Anthropic stabilisce quali comandi esistono (leggi, scrivi, elenca…) e come si chiamano; dove e come i file vengono salvati davvero (su disco, in un database, nel cloud) lo decide lo sviluppatore.

→ Pre-corso: API e REST (cos'è un'interfaccia, il "contratto" tra programmi).

Quale usare?

- Un loop con decine di risultati prolissi degli strumenti → **modifica del contesto**.
- Thinking esteso e voglia di mantenere alte le cache → **modifica del contesto** con `clear_thinking`.
- Conversazione lunga di ricerca dove la narrazione va conservata → **compattazione**.
- Risultati che devono sopravvivere tra sessioni diverse → **strumento di memoria**.
- Numeri che non devono mai essere parafrasati → **blocco dei fatti** (il metodo più sicuro di tutti).

---

8. Limiti e rifiuti

`max_tokens` e il budget dell'attività

`max_tokens` è il limite massimo di lunghezza della risposta. È un limite che **il modello non vede**: quando lo raggiunge, il testo si interrompe a metà, e il lavoro si perde.

Il **budget dell'attività** (`output_config.task_budget`, con un minimo di 20.000 token) è invece un limite che **il modello vede**. Così può regolare il ritmo e concludere in modo ordinato, come un cronometro in una gara. Non è disponibile su Sonnet 5.

Un'analogia: `max_tokens` è come staccare la corrente a un oratore quando scade il tempo, a metà frase. Il budget dell'attività è come mostrargli un cartello "mancano 5 minuti": può chiudere il discorso in modo ordinato.

I rifiuti

Quando Claude rifiuta una richiesta, la risposta arriva con stato HTTP `200` (tutto ok) e con `stop_reason: "refusal"`. Se il codice legge il contenuto senza controllare lo stato, il rifiuto diventa una risposta vuota presa per buona.

È la stessa classe di errore del dominio 2: scambiare un fallimento per un risultato. **Controlla sempre `stop_reason` prima di leggere il contenuto.**

→ Pre-corso: Richiesta HTTP. Il codice `200` dice solo che la comunicazione è andata a buon fine, non che Claude abbia fatto ciò che era stato chiesto.

---

9. Le sette cose da ricordare

1. I fatti critici vanno in un blocco persistente, non lasciati al riassunto.
2. Il sentiment non determina l'escalation: conta la complessità.
3. Un errore deve distinguere "nessun risultato" da "ricerca non eseguita".
4. L'accuratezza va suddivisa per tipo di documento e per campo.
5. La confidenza va calibrata, altrimenti è un numero senza significato.
6. Conflitti annotati, date sempre esplicite, nessuna fusione di fonti.
7. Nella compattazione, passa l'intero `response.content`.

Per capire meglio

Il filo rosso del dominio è sempre lo stesso: **non far perdere o deformare l'informazione**. Quando una domanda parla di numeri esatti che sbiadiscono, la risposta è il blocco dei fatti. Quando parla di tono arrabbiato e problema semplice, la risposta è non escalare. Quando parla di un servizio spento che restituisce una lista vuota, la risposta è distinguere i due casi. E quando parla di compattazione, ricordati: intero `response.content`.

Tre ragionamenti tipici

"Dopo una conversazione lunga, l'agente propone un rimborso di 150 $invecedi 149,99$." Il riassunto ha arrotondato l'importo. La correzione è un blocco dei fatti persistente con i valori esatti, non un riassunto "più accurato".

"Il sistema di estrazione ha il 97% di accuratezza, ma i reclami aumentano." La media nasconde una categoria che va male. Bisogna misurare l'accuratezza per tipo di documento e per campo, e concentrare la revisione umana dove serve.

"Il report finale riporta un fatturato senza dire che due fonti davano valori diversi." Il conflitto è stato risolto in silenzio. La correzione è conservare provenienza e date di ogni affermazione e annotare esplicitamente il disaccordo.